

C++ Template 延伸自主練習

1. 自主學習主題與目標

本次自主學習的主題為「**C++ Template** 與 **Standard Template Library (STL)**」。
透過本次學習，我希望達成以下目標：

1. 理解 Template 的概念與用途
2. 熟悉 STL 常見容器 (vector、stack、queue、map)
3. 能夠使用 STL 實作基本資料管理系統
4. 提升程式設計效率與可讀性

2. Template 與 STL 的關係

模板 (Template) 是指函式模板與類模板，是一種參數化類型機制，可以讓程式碼適用於多種資料型別。STL (Standard Template Library) 正是建立在 Template 之上的函式庫。

例如：

```
vector<int>  
stack<char>
```

STL 中的所有容器與演算法 (如 sort、find) 都是透過 template 實現，因此可以適用於各種資料型別。

3. 常見 STL 容器用途整理

vector:

動態調整陣列大小，遵循陣列規則，可以排序與搜尋

quene:

遵循先進先出(FIFO)，類似於排隊，後輸入的元素會被排在後方，同時可快速搜尋首尾的元素

stack:

遵循後進先出(LIFO)，類似於堆疊，越後輸入的元素會越接近第一位，同時優先被取出或刪除

map:

key-value結構，可以給予元素(key)數值(value)，對資料進行整理

4. 第 2 週小專題設計說明

本專題為「學生資料管理系統」，主要功能如下：

1. 新增學生資料
2. 列出所有學生
3. 依成績排序
4. 查詢學生
5. 成績統計

資料結構選擇：

使用 **vector** 儲存學生資料

5. 主要程式架構與重要程式片段

(1) 結構定義:

```
struct Student
{
    string id;
    string name;
    int score;
};
```

(2) 排序:

```
void sortStudents()
{
    sort(students.begin(), students.end(),
        [](const Student &a, const Student &b)
        {return a.score > b.score;});

    cout << "已依成績排序\n";
}
```

(3) 查詢功能:

```
for (auto &s : students) {
    if (s.id == id) {
        cout << "找到："
            << s.id << " (const char [2])" "
            << s.name << " "
            << s.score << endl;
        return;
    }
}
```

(4) 統計功能:

```
for (auto &s : students) {
    sum += s.score;

    if (s.score > maxScore) maxScore = s.score;
    if (s.score < minScore) minScore = s.score;

    if (s.score >= 60) pass++;
    else fail++;
}
```

6. 執行畫面或輸出結果

本專題為「學生資料管理系統」，主要功能如下：

1. 新增學生資料
2. 列出所有學生
3. 依成績排序
4. 查詢學生
5. 成績統計

使用 **Add student** 新增我的學生資料，再用 **List all student** 列出我的資料

```
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
選擇：█
```

```
選擇：1
輸入學號 姓名 成績：11459045 楊景任 90
新增成功！
```

```
選擇：2
學號 姓名 成績
11459045 楊景任 90
```

7. GitHub repo link 與檔案結構說明

Github連結:<https://github.com/YJJOuOb/Final-report1>

檔案結構:

Final report1

- main.cpp
- README.md
- report.pdf

8.學習心得與未來可延伸方向

透過本次學習，我對C++ STL 有了更深入的理解，尤其是：

- vector 的動態管理能力

- sort 的進階應用

- map 在資料查詢上的優勢

未來可以延伸的方向：

- 使用 unordered_map 提升效能

- 加入資料儲存(檔案 I/O)

- 開發圖形化介面(GUI)

本次學習讓我理解到STL 在實做上的重要性，也提升了解決問題的能力。